

AGENTICT²S: Robust Text-to-SPARQL via Agentic Collaborative Reasoning over Heterogeneous Knowledge Graphs for the Circular Economy

Yang Zhao* Chengxiao Dai* Wei Zhuo Chuan Fu Tan Yue Xiu Dusit Niyato
Jonathan Low Zhaozhi Eugene H.Z. Ho Daren Tan Zong Loong

Abstract

Question answering over heterogeneous knowledge graphs (KGQA) involves reasoning across diverse schemas, incomplete alignments, and distributed data sources. Existing text-to-SPARQL approaches rely on large-scale domain-specific fine-tuning or operate within single-graph settings, limiting their generalizability in low-resource domains and their ability to handle queries spanning multiple graphs. These challenges are particularly relevant in domains such as the circular economy, where information about classifications, processes, and emissions is distributed across independently curated knowledge graphs (KGs). We present AGENTICT²S, a modular framework that decomposes KGQA into subtasks managed by specialized agents responsible for retrieval, query generation, and verification. A scheduler assigns subgoals to different graphs using weak-to-strong alignment strategies. A two-stage verifier detects structurally invalid and semantically underspecified queries through symbolic validation and counterfactual consistency checks. Experiments on real-world circular economy KGs demonstrate that AGENTICT²S improves execution accuracy by 17.3% and triple level F_1 by 25.4% over the best baseline, while reducing the average prompt length by 46.4%. These results demonstrate the benefits of agent-based schema-aware reasoning for scalable KGQA and support decision-making in sustainability domains through robust cross-graph reasoning.

Introduction

Large language models (LLMs) demonstrate strong performance in a range of natural language processing tasks (Min et al. 2023), but face limitations in knowledge-intensive settings. Because their outputs are based on pre-trained parameters, LLMs are prone to hallucination and may lack factual consistency (Ji et al. 2023). Retrieval-augmented generation (RAG) addresses this by incorporating external information at inference time (Lewis et al. 2020), although most RAG systems operate over unstructured text, which lacks the semantic structure required for accurate multi-hop reasoning. These limitations are critical in the circular economy, where answering queries requires reasoning over multiple heterogeneous KGs with inconsistent schemas.

Structured retrieval methods such as GraphRAG (Ji et al. 2024) have been proposed to integrate LLMs with knowl-

edge graphs (KGs), which offers improved precision and interpretability. However, existing systems often embed large subgraphs in prompts (Cui, Sun, and Hu 2024), increasing latency and exceeding context limits; rely on weakly supervised retrievers (Yasunaga et al. 2021); and couple retrieval, generation and validation into single-agent pipelines (He et al. 2024; Xu et al. 2024), reducing modularity and adaptability. In addition, even state-of-the-art models struggle to generate semantically correct SPARQL queries, particularly on unseen graphs (Meyer et al. 2024). SPARQL is a standardized query language for RDF-based KGs (W3C 2013), used to express structured queries involving logical conditions, joins, and filters.

This work introduces AGENTICT²S, a modular framework for answering questions on heterogeneous KGs. The system decomposes a natural language query into sub-goals corresponding to reasoning operations such as entity lookup or comparison. **Each subgoal is assigned to candidate graphs via schema alignment and retrieval history, translated into executable SPARQL using schema-guided decoding, and validated using symbolic checks and counterfactual consistency analysis.** The architecture separates retrieval, generation, and verification into distinct agents to improve transparency, error isolation, and generalizability.

To the best of our knowledge, this is the first formulation of text-to-SPARQL generation for KGQA over heterogeneous graphs as a multi-agent problem. The system consists of five agents, including subgoal decomposer, retriever allocator, SPARQL generator, dual-stage verifier, and result integrator, that coordinate to support compositional reasoning and robust query execution. The design is evaluated in the context of the circular economy, where reasoning over distributed emissions data and non-unified classification schemes requires schema-aware planning and verifiable synthesis (Atiku 2020).

Contributions.

- This work formulates cross-graph question answering over heterogeneous KGs as a modular multi-agent process. The design addresses the limitations of monolithic text-to-SPARQL pipelines by separating retrieval, synthesis, and verification into distinct composable agents. This decomposition allows for interpretable reasoning, schema-level adaptability, and improved fault isolation.

*Equal contribution.

- The framework incorporates three key components: (i) a weak-to-strong alignment allocator for schema-driven graph selection, (ii) a pattern-based SPARQL synthesizer with template scaffolding and post-hoc grammar repair, and (iii) a dual-stage verifier combining symbolic validation and counterfactual consistency checks. These are instantiated as five specialized agents with different roles.
- Experimental results in three real-world circular economy KGs show that the proposed method improves execution accuracy by 17.3% and triple level F_1 by 25.4% over the best baseline, while reducing the average prompt length by 46.4%.

Related Work

RAG methods incorporate external information to improve the factual consistency of the language model output (Lewis et al. 2020; Gao et al. 2023). While effective for unstructured question answering, they assume flat input formats and are not well suited for tasks involving structured queries, multi-hop reasoning, or schema constraints.

GraphRAG (Edge et al. 2024; Wu et al. 2024a) and related methods extend RAG by converting local graph neighborhoods into textual prompts. These methods retain some structural context but are limited by prompt length, formatting dependencies, and an assumption of a single underlying graph. Most also lack validation mechanisms for generated queries (Wang et al. 2024).

Other work decomposes question answering into modular components for retrieval, reasoning, and execution (Tran et al. 2025; Barbosa, Santos, and Novais 2024). Multi-agent systems extend this idea by assigning subtasks to separate agents (Guo et al. 2024; Yang et al. 2025). For example, ReAgent (Zhao et al. 2025) applies local and global backtracking in multi-hop QA over text. These systems are typically applied to unstructured input and do not address structured query generation.

General purpose agent frameworks such as ReAct (Yao et al. 2023), AutoGen (Wu et al. 2024b), CAMEL (Li et al. 2023), and MetaGPT (Hong et al. 2023) implement role-based decomposition for planning and tool interaction across tasks. These methods have been applied to information retrieval, decision-making, and code synthesis in unstructured or semi-structured settings. They are not designed for structured KGQA and lack support for schema-aware reasoning or symbolic query verification.

Problem Formulation

Given a natural language question q , the task is to generate an executable SPARQL query s that retrieves a complete and semantically correct answer set A from a subset of a heterogeneous KG collection $\mathcal{G} = \{G_1, G_2, \dots, G_n\}$. Each KG $G_i \in \mathcal{G}$ consists of entities and relations represented as triples (h, r, t) , where h and t are entities, and r is a relation type (Yih et al. 2015; Saxena, Tripathi, and Talukdar 2020).

A valid SPARQL query s is expected to satisfy two properties. First, it must be executable, meaning it can be evaluated on at least one graph $G_i \in \mathcal{G}$ without syntax or schema violations. Second, it must be semantically accurate, in that

it reflects the intent of the question q and returns a complete and correct answer set $A \subseteq \text{Ent}$, where Ent is the union of entity sets across all graphs in $\mathcal{G}$.

This task presents several challenges. The graphs in $\mathcal{G}$ may differ in schema, vocabulary, and structural representation, introducing substantial heterogeneity. Answering a question may require distributed reasoning, where information must be aggregated from multiple KGs. Furthermore, many KGQA benchmarks lack the gold-standard SPARQL annotations, limiting the feasibility of supervised learning. Finally, practical systems must meet efficiency requirements, making retrieval and query generation time sensitive.

The objective is to develop a system that, given an input question q and a collection $\mathcal{G}$ of KGs, produces a query s such that executing s on the appropriate subset of $\mathcal{G}$ yields a correct and complete answer set A .

System Design

We present AGENTICT²S, a modular multi-agent framework to answer natural language questions on heterogeneous KGs. Unlike end-to-end GraphRAG pipelines (Han et al. 2024), which tightly link retrieval and generation in a single prompt-based step, AGENTICT²S separates the task into distinct components: subgoal decomposition, adaptive retrieval, query generation, verification, and answer aggregation. Each component is implemented as an independent agent with a clear input-output interface (Wu et al. 2025), allowing for parallelism, interpretability, and easier adaptation across domains. Given a question, the system first decomposes it into a sequence of subgoals. Each subgoal is then routed to a target KG selected based on schema compatibility and retrieval utility. Structured queries are generated under schema constraints and validated through a two-stage verification process. The resulting sub-answers are aggregated into a final response. The complete workflow is detailed in Algorithm 1.

Algorithm 1: AGENTICT²S Pipeline for Compositional KGQA

Require: Natural language question q , KGs $\mathcal{G} = G_1, G_2, \dots, G_n$

Ensure: Final answer set A

```

1:  $g_1, g_2, \dots, g_k \leftarrow \text{SUBGOAL\_PARSER}(q)$ 
2: for each subgoal  $g_i$  do
3:    $\mathcal{R}_i \leftarrow \text{CANDIDATE\_RETRIEVERS}(g_i, \mathcal{G})$ 
4:    $G_i \leftarrow \text{HIERARCHICAL\_ALLOCATOR}(g_i, \mathcal{R}_i)$ 
5:    $q_i \leftarrow \text{SPARQL\_SYNTHESIZER}(g_i, G_i)$ 
6:   if  $\text{VERIFIER}(q_i, G_i)$  passes then
7:      $A_i \leftarrow \text{EXECUTE\_QUERY}(q_i, G_i)$ 
8:   else
9:      $A_i \leftarrow \text{FALLBACK\_OR\_SKIP}(g_i)$ 
10:  end if
11: end for
12:  $A \leftarrow \text{CONSENSUS\_AGGREGATOR}(A_1, A_2, \dots, A_k)$ 
13: return  $A$ 
```

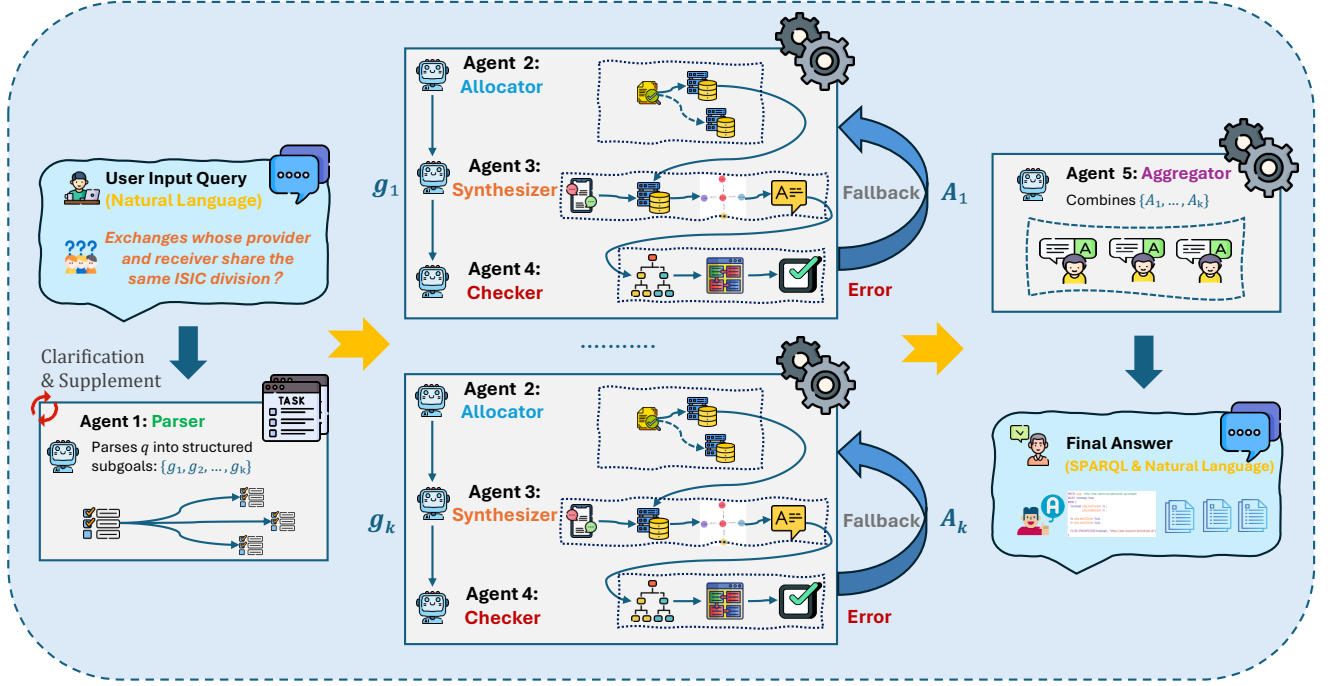


Figure 1: Overview of the AGENTICT²S framework. The system decomposes a natural language query into subgoals, allocates them to relevant graphs, generates SPARQL queries, verifies their correctness, and integrates the answers.

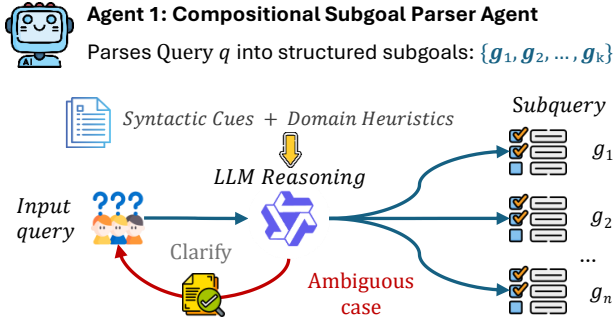


Figure 2: The overall workflow of the Compositional Subgoal Parser Agent.

Compositional Subgoal Parser Agent

The **Compositional Subgoal Parser Agent** initiates the AGENTICT²S pipeline (Algorithm 1, line 1) by decomposing a natural language question into a sequence of subgoals, each corresponding to a basic reasoning operation such as entity retrieval, condition filtering, or aggregation (Figure 2). The parser employs a rule-based strategy that combines shallow syntactic analysis with domain-specific heuristics. Structured prompts are used to guide a language model in identifying syntactic patterns, segmenting the question into subgoals, and assigning intent labels based on linguistic structure. The agent also includes mechanisms to detect ambiguity or underspecified semantics; when triggered, these

prompt the user for clarification to ensure subgoal completeness and correctness.

This approach does not require task-specific training data and is generalized across heterogeneous KGs. By decomposing complex queries into semantically independent components, the parser simplifies the reasoning process and enhances coordination among downstream agents.

Hierarchical Alignment Allocator Agent

Each subgoal is assigned to a relevant KG by the **Hierarchical Alignment Allocator Agent** (Algorithm 1, line 4), which selects candidate graphs through a two-stage retrieval process: coarse-grained filtering and fine-grained schema alignment based on the subgoal’s intent and constraints (Figure 3).

In the **weak retrieval** stage, the allocator conducts a vector similarity search over embedded metadata, including schema summaries and domain labels, to identify a subset of candidate KGs. For each selected graph, a secondary search is conducted over its internal source files, such as ontology documents and schema descriptions, to locate relevant terms associated with the subgoal. In the **strong retrieval** stage, schema-level alignment is performed on the top-ranked candidates. This step extracts predicates, class hierarchies, and domain-range constraints to assess whether the graph’s structure is compatible with the subgoal.

The KG pool and metadata index support incremental updates, allowing the system to incorporate new resources without reprocessing. As additional KGs and metadata become available, the allocator adjusts its retrieval strategy to maintain schema alignment. Retrieval decisions are further

refined using utility scores from downstream verification, enabling the system to respond to schema drift and improve retrieval precision in complex queries.

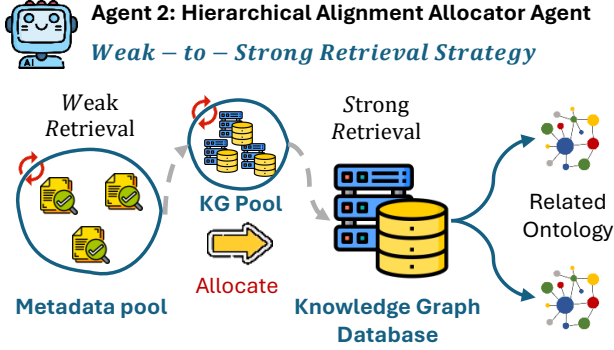


Figure 3: The overall workflow of the Hierarchical Alignment Allocator Agent.

Pattern-Driven SPARQL Synthesizer Agent

The **Pattern-Driven SPARQL Synthesizer Agent** (Algorithm 1, line 5) generates executable SPARQL queries from verified subgoals in three stages: query skeleton design, schema grounding, and post-hoc decoding (Figure 4).

In the **query skeleton design** stage, the agent selects a query template from a predefined library based on the subgoal’s intent, such as entity retrieval, predicate filtering, or attribute matching. Each template represents an abstract SPARQL pattern and is parameterized with schema elements obtained from the previous agent. The template is instantiated with these elements to produce an initial query structure that is consistent with the schema and semantics of the target KG.

The **schema grounding** stage refines this structure by incorporating ontology-level constraints, including predicate types, class hierarchies, and domain-range specifications. This step replaces generic placeholders with concrete schema components and enforces compatibility between entities, relations, and types defined in the graph. The resulting query is semantically coherent and structurally valid within the context of the selected KG.

To ensure reliability under diverse query conditions, the final stage, **post-hoc decoding**, serves as a model-agnostic validation layer that enforces structural and semantic correctness. Operating independently of the generation model, it systematically verifies syntax, resolves prefixes, and checks predicate usage, variable bindings, and schema alignment. In addition, it identifies query structures that are prone to execution failure, such as those that produce empty answer sets or violating ontology constraints. By applying targeted corrections only when necessary, post-hoc decoding provides a final safeguard, producing SPARQL queries that are not only valid and executable, but also resilient to schema variability across KGs.

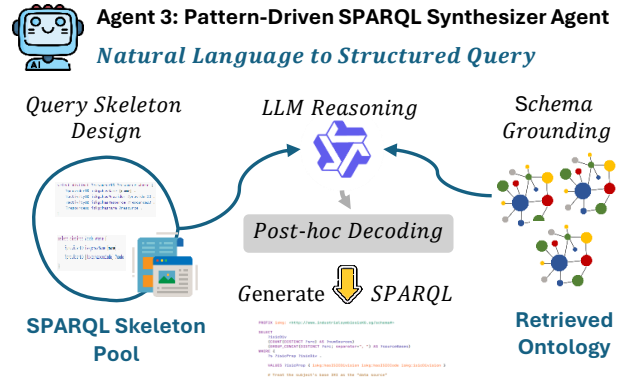


Figure 4: The overall workflow of the Pattern-Driven SPARQL Synthesizer Agent.

Dual-Stage Consistency Checker Agent

To safeguard against execution errors and underspecified queries, the **Consistency Checker Agent** (Algorithm 1, line 6) applies a two-stage verification process prior to query execution. This agent functions as a quality control layer that filters out syntactically invalid or semantically ambiguous queries before they reach the execution stage.

In the first stage, symbolic validation is applied to assess syntactic correctness, predicate usage, and type compatibility with the schema of the target KG. The agent also performs a preliminary execution check to ensure that the query yields a non-empty result under current graph conditions.

In the second stage, counterfactual testing is used to evaluate the semantic specificity. The agent systematically perturbs the selected query components, such as entities, filters, or predicates, and compares the result sets. Queries that produce unchanged outputs across such perturbations are flagged as overly general or underspecified. Only queries that pass both checks are forwarded for execution; others are revised or discarded. This mechanism improves the reliability of the final result by enforcing both structural validity and semantic precision.

Multi-Graph Consensus Aggregator Agent

Consensus Aggregator Agent (Algorithm 1, line 12) consolidates partial query results retrieved from multiple KGs. The agent first performs entity alignment across heterogeneous sources to resolve equivalence and redundancy. It then generates a unified natural language response by merging sub-answers using an LLM. This aggregation step ensures that the final output is consistent across sources and robust to variations in schema or terminology.

Complexity Analysis

For a query decomposed into k sub-goals, the end-to-end work of Algorithm 1 is $O(k) [|q| + n + \sum_{j=1}^p |\mathcal{S}_j| + BL + m T_{\text{exec}} + |G_j|]$, where $|q|$ is the question length, n the total number of KGs, $p \ll n$ the candidates that survive weak-tier retrieval, $\mathcal{S}_j$ the schema slice of graph G_j , B and L the beam width and decoded length in the synthesiser, m the

counter-factual checks in the verifier, and T_{exec} the cost of one SPARQL evaluation on the executed graph of size $|G_j|$. All stages are embarrassingly parallel across sub-goals and graphs, so wall-clock latency is dominated by a handful of LLM calls and at most k verified query executions.

Experiments

Experimental Setup

We evaluate AGENTICT²S and all baselines on a standardized text-to-SPARQL generation task, where each system receives a natural language question and generates a SPARQL query, which is executed against the relevant KGs. AGENTICT²S runs in a hybrid setup that combines local model inference with API-based access to LLMs. The system is implemented using the LangGraph framework (LangGraph Contributors 2024), which supports agent coordination, intermediate state tracking, and structured execution flow.

Local components include lightweight models for schema retrieval, subgoal parsing, and triple pattern scoring, executed on a workstation with four NVIDIA RTX 4090 Ti GPU. For language generation tasks that require larger context and structured decoding, we use LLaMA3-70B and LLaMA3.3-70B (Touvron et al. 2024), Qwen3-32b and QwQ-32b (Qwen Team 2024) via Groq API (Groq, Inc. 2024). The remaining models are accessed via their official APIs. All models are evaluated in zero-shot mode with a fixed temperature of 0.7 and no task-specific fine-tuning.

Datasets and Evaluation Settings

We evaluated three RDF KGs related to industrial resource exchanges in the circular economy domain, all curated in-house. Each KG differs in schema complexity and coverage.

- **German-IS.** This dataset (8,649 triples) models 265 exchange cases from a national German program. Entities represent industrial actors and resources, annotated with multiple classification codes.
- **EU-Pilot.** This dataset (11,810 triples) covers 305 cases from a European project. It includes structured metadata and is annotated with standardized waste classification codes.
- **Waste-Ledger.** This dataset (52,402 triples) contains 1,727 instances of waste and resource flows across Europe, including quantitative attributes.

The benchmark consists of 328 queries spanning both single-KG and cross-KG settings, curated from three circular economy knowledge graphs. The models are evaluated in three random seeds; we report mean $\pm$ standard deviation.

Baselines

We compare against three categories of baselines:

- **Zero-Shot LLMs.** Models prompted to generate SPARQL from natural language, without task-specific fine-tuning. This includes LLaMA3-70B and LLaMA3.3-70B (Touvron et al. 2024), Qwen3-32b and QwQ-32b (Qwen Team 2024), DeepSeek-V3 (Liu et al. 2024), and GPT-4o (OpenAI 2024).

- **Reasoning-Augmented LLMs.** The same models are evaluated with chain-of-thought s prompting (Wei et al. 2022) and self-ask prompting (Press et al. 2022) to elicit intermediate reasoning steps prior to query generation.
- **Agent Controllers.** General-purpose multi-agent frameworks, including ReAct (Yao et al. 2023) and AutoGen (Wu et al. 2024b), are combined with LLaMA3 and Qwen3. These systems define planning and execution roles, but do not incorporate symbolic verification or schema-guided generation.

Evaluation Metrics

We evaluated model performance using four metrics: execution accuracy (EA), query syntax correctness (QSC), triple-level F1 (TF1), and average token usage (ATU). EA measures whether the generated SPARQL query executes without error and returns an answer set that matches the gold standard. QSC reflects the proportion of syntactically valid queries based on Apache Jena parsing. TF1 computes micro-averaged F1 over triple patterns in the WHERE clause, ignoring variable renaming and FILTER expressions. ATU reports the average number of tokens used per prediction as a proxy for generation efficiency.

Main Results

Table 1 reports the performance of all baselines and agent-based systems across EA, QSC, TF1, and ATU. AGENTICT²S outperforms all models, achieving 72.43% EA, 76.21% QSC, and 83.68% TF1. These correspond to relative gains of 17.3% in EA and 25.4% in TF1 over the strongest LLM baseline, V-L-GPT-4o, which reaches 55.15% EA and 58.32% TF1.

Token Efficiency. Despite using structured decoding and verification, AGENTICT²S remains efficient, averaging 875 tokens per example. This represents a 46.4% reduction in token usage compared to V-LLaMA3.3-70B (1632 tokens), while achieving higher performance across all metrics.

Comparison to Reasoning-Augmented LLMs. Among chain-of-thought-based baselines, R-L-DEEPSEEK-R1 achieves the best results (50.32% EA, 52.18% TF1) but remains 22% behind our framework in EA. This highlights the limitations of prompting alone in capturing structural and semantic constraints across KGs.

Knowledge Graph Allocation. We evaluate the allocator’s effectiveness by measuring the proportion of subgoals routed to a graph that actually contains the gold answer. As shown in Figure 5, AGENTICT²S attains the highest allocation accuracy, exceeding the strongest baseline (DeepSeek-R1) by over 10%. This precision directly impacts EA: correctly routed subgoals are more likely to match relevant schemas, leading to higher query validity and fewer empty results.

Cross-KG Robustness. Tables 2 and Table 3 present the EA and QSC scores per graph. AGENTICT²S consistently leads across all three graphs. On German-IS and EU-Pilot, gains are attributed to improved subgoal decomposition and

Table 1: Performance comparison of different model configurations on structured KGQA. AGENTICT²S achieves the highest accuracy across all metrics while maintaining lower token usage.

Model	EA (%)	QSC (%)	TF1 (%)	ATU
V-LLaMA3-70B	10.12 $\pm$ 2.45	15.47 $\pm$ 3.12	18.36 $\pm$ 2.78	1632 ($\pm$ 110)
V-LLaMA3.3-70B	12.28 $\pm$ 2.13	18.65 $\pm$ 3.08	10.52 $\pm$ 2.33	
V-L-GPT-4o	55.15 $\pm$ 1.31	60.04 $\pm$ 0.56	58.32 $\pm$ 1.12	
V-L-DeepSeek-V3	45.74 $\pm$ 1.22	50.19 $\pm$ 1.41	47.09 $\pm$ 1.13	
R-Qwen3-32B	40.25 $\pm$ 1.05	45.68 $\pm$ 1.82	42.72 $\pm$ 1.01	1632 ($\pm$ 110)
R-QwQ-32B	35.67 $\pm$ 1.01	40.31 $\pm$ 0.75	38.25 $\pm$ 0.89	
R-L-DeepSeek-R1	50.32 $\pm$ 1.14	55.21 $\pm$ 1.22	52.18 $\pm$ 0.07	
A-ReAct+LLaMA3-70B	28.52 $\pm$ 1.03	37.33 $\pm$ 1.74	33.25 $\pm$ 0.98	1518 ($\pm$ 110)
A-ReAct+Qwen3-32B	47.13 $\pm$ 1.45	54.65 $\pm$ 2.01	52.30 $\pm$ 1.88	
A-AutoGen+LLaMA3-70B	31.21 $\pm$ 2.17	33.02 $\pm$ 0.89	37.14 $\pm$ 1.06	
A-AutoGen+Qwen3-32B	49.06 $\pm$ 1.12	58.39 $\pm$ 1.93	53.28 $\pm$ 1.62	
Ours: AGENTICT²S	72.43 $\pm$ 1.24	76.21 $\pm$ 1.19	83.68 $\pm$ 0.39	875 $\pm$ 90

Note: V = Vanilla LLM, R = Reasoning LLM, L = Large-Scale Pretrained Model, A = Agent Framework.

Table 2: EA on single-KG.

Model	German-IS	EU-Pilot	Waste-Ledger
V-LLaMA3-70B	15.08 $\pm$ 1.86↓	16.05 $\pm$ 1.19↓	13.82 $\pm$ 2.75↓
V-LLaMA3.3-70B	17.21 $\pm$ 1.43↓	14.31 $\pm$ 0.80↓	14.28 $\pm$ 0.76↓
V-L-GPT-4o	76.50 $\pm$ 2.93↑	74.24 $\pm$ 1.03↑	72.05 $\pm$ 0.82↑
V-L-DeepSeek-V3	69.76 $\pm$ 2.63↓	68.22 $\pm$ 1.54↓	63.56 $\pm$ 2.36↓
R-Qwen3-32B	53.12 $\pm$ 2.64↓	54.55 $\pm$ 2.08↓	49.67 $\pm$ 0.67↓
R-QwQ-32B	56.09 $\pm$ 2.88↓	53.73 $\pm$ 1.43↓	51.82 $\pm$ 2.03↓
R-L-DeepSeek-R1	79.63 $\pm$ 2.67↑	74.86 $\pm$ 1.52↑	72.91 $\pm$ 0.94↑
Ours: AGENTICT²S	85.64 $\pm$ 1.06↑	87.72 $\pm$ 1.28↑	85.01 $\pm$ 2.33↑

Table 3: QSC on single-KG.

Model	German-IS	EU-Pilot	Waste-Ledger
V-LLaMA3-70B	21.31 $\pm$ 0.75↓	25.12 $\pm$ 0.97↓	25.83 $\pm$ 0.60↓
V-LLaMA3.3-70B	27.48 $\pm$ 2.00↓	24.44 $\pm$ 1.72↓	24.35 $\pm$ 1.55↓
V-L-GPT-4o	86.55 $\pm$ 1.24↑	88.92 $\pm$ 1.07↑	86.44 $\pm$ 1.57↑
V-L-DeepSeek-V3	61.39 $\pm$ 1.80↓	67.23 $\pm$ 0.58↓	62.64 $\pm$ 1.90↓
R-Qwen3-32B	75.44 $\pm$ 1.28↓	73.64 $\pm$ 1.00↓	75.86 $\pm$ 1.53↓
R-QwQ-32B	67.49 $\pm$ 1.12↓	72.98 $\pm$ 0.84↓	68.28 $\pm$ 0.68↓
R-L-DeepSeek-R1	86.05 $\pm$ 0.81↑	87.75 $\pm$ 1.46↑	85.22 $\pm$ 1.89↑
Ours: AGENTICT²S	97.14 $\pm$ 1.62↑	96.99 $\pm$ 0.55↑	97.91 $\pm$ 1.44↑

schema grounding, which enhance EA. On Waste-Ledger, which includes quantitative predicates and deeper ontologies, QSC gains reflect the effect of post-hoc decoding and structural validation.

These results suggest that modular reasoning and symbolic validation, particularly subgoal decomposition and verification, are critical to scaling text-to-SPARQL to diverse and schema-rich KGs.

Ablation Study

To assess the contribution of each component, we ablate the following modules:

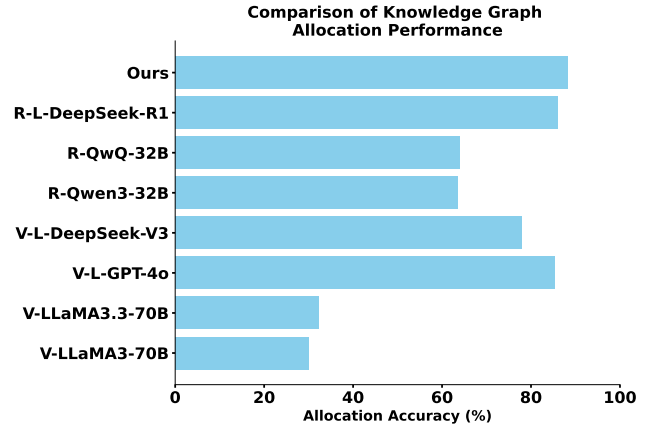


Figure 5: Comparison of knowledge graph allocation performance across models.

- **w/o Decomposer.** Disables subgoal parsing; the full question is passed directly to the SPARQL generator.
- **w/o Allocator.** Replaces graph selection with BM25 retrieval over the full KG pool.
- **w/o Synthesizer.** Skips query generation; retrieved schema triples are returned as pseudoanswers.
- **w/o Verifier.** Executes SPARQL queries without symbolic or counterfactual validation.

All variants are evaluated on three random seeds, and the results are reported as mean $\pm$ standard deviation.

As shown in Table 4, removing the subgoal decomposer leads to a sharp drop in EA from 72.4% to 23.7%, underscoring the importance of compositional parsing. Disabling the allocator similarly reduces performance, reflecting the impact of semantic drift when retrieval is not schema-aware. Omitting the verifier results in a 17% decline in EA, indicat-

Table 4: Ablation study on the contribution of key components in the AGENTICT²S framework. Results reported on the full evaluation set.

Configuration	EA (%)	QSC (%)	TF1(%)
Ours w/o Decomposer Agent	23.68 $\pm$ 0.66	45.31 $\pm$ 0.19	45.42 $\pm$ 1.32
Ours w/o Allocator Agent	25.12 $\pm$ 0.18	31.02 $\pm$ 1.12	37.08 $\pm$ 0.36
Ours w/o Synthesizer Agent	—	—	—
Ours w/o Verifier Agent	55.12 $\pm$ 0.56	60.23 $\pm$ 1.10	66.17 $\pm$ 0.23
Ours w/o Agentic Collaborative	43.43 $\pm$ 0.78	52.56 $\pm$ 0.50	47.22 $\pm$ 0.16
Full AGENTICT²S	72.43 $\pm$ 1.24	76.21 $\pm$ 1.19	83.68 $\pm$ 0.39

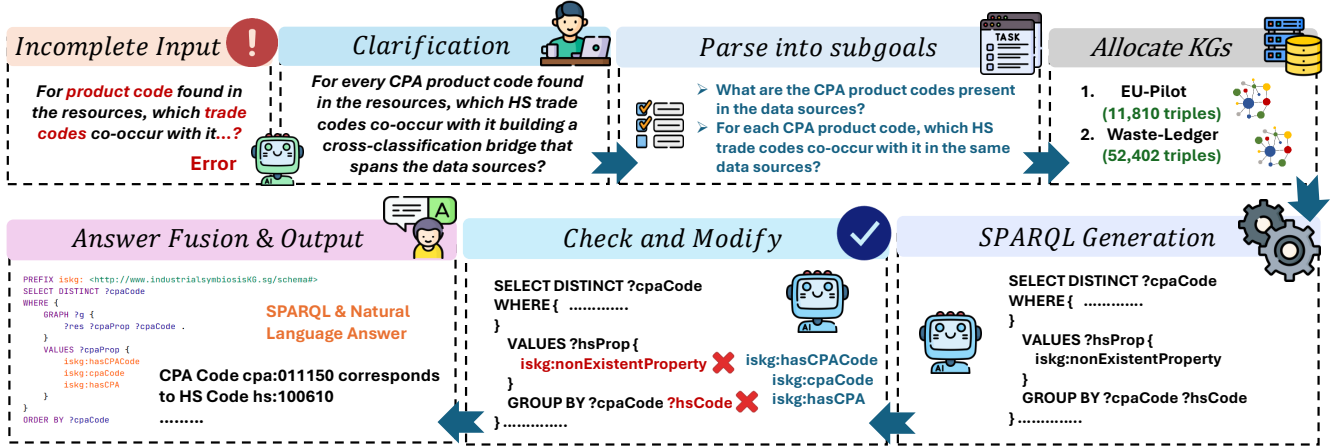


Figure 6: Example workflow of AGENTICT²S applied to a circular economy query.

ing that many syntactically valid queries are semantically underspecified. These results confirm the role of modular reasoning and symbolic validation in achieving accurate and robust SPARQL generation.

Qualitative Analysis

Figure 6 provides a step-by-step example of query processing under the AGENTICT²S framework. The initial input is incomplete and underspecified: “For product code found in the resources, which trade codes co-occur with it?” The system initiates a clarification step, reformulating the input into a fully specified intent that references waste classification codes and requests cooccurrence information.

The clarified question is decomposed into subgoals, which are dispatched to the appropriate KGs (EU-Pilot and Waste-Ledger) based on schema compatibility. SPARQL queries are then generated for each subgoal using pattern-based templates. During verification, a syntactic and semantic check identifies a non-existent predicate in the initial output. The system modifies the query accordingly, replacing the faulty predicate with the correct one.

Following validation, verified queries are executed and their outputs aggregated. The final output includes both the SPARQL representation and a natural language statement indicating the mapping between CPA code 011150 and HS code 100610. The example illustrates the role of clarification, decomposition, schema validation, and multi-graph

answer fusion in the overall pipeline.

Conclusion

We have presented AGENTICT²S, a multi-agent framework for text-to-SPARQL generation over heterogeneous KGs. The modular architecture decomposes the task into subgoal parsing, graph allocation, schema-aware synthesis, verification, and aggregation, enabling compositional reasoning across diverse schemas. Then, the combination of grammar-constrained decoding with post-hoc repair ensures high query executability, while modularity enables effective scaling. Evaluation on three circular economy datasets demonstrates improvements in execution accuracy, query validity, and prompt efficiency over strong baselines. The results support the use of multi-agent designs for structured question answering in settings involving multiple graphs, variable ontologies, and domain-specific constraints. Beyond technical performance, AGENTICT²S provides a practical tool for decision-making in the circular economy. Its ability to compose and validate SPARQL queries across fragmented KGs enables stakeholders, such as policy analysts, sustainability officers, and industrial planners, to answer complex questions about resource reuse, emissions trade-offs, and regulatory alignment. By surfacing structured insights from heterogeneous datasets, the framework supports data-informed actions in sustainability reporting, materials tracking, and circular supply chain optimization.

References

- Atiku, S. O. 2020. Knowledge Management for the Circular Economy. In *Handbook of Research on Entrepreneurship Development and Opportunities in Circular Economy*, 520–537. IGI Global.
- Barbosa, R.; Santos, R.; and Novais, P. 2024. Collaborative Problem-solving with LLM: a Multi-agent System Approach to Solve Complex Tasks Using Autogen. In *International Conference on Practical Applications of Agents and Multi-Agent Systems*, 203–214. Springer.
- Cui, Y.; Sun, Z.; and Hu, W. 2024. A Prompt-based Knowledge Graph Foundation Model for Universal In-Context Reasoning. *Advances in Neural Information Processing Systems*, 37: 7095–7124.
- Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R. O.; and Larson, J. 2024. From Local to Global: A Graph RAG Approach to Query-focused Summarization. *arXiv preprint arXiv:2404.16130*.
- Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, H.; and Wang, H. 2023. Retrieval-augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*, 2(1).
- Groq, Inc. 2024. Groq API and LPU Inference Platform. <https://groq.com>. Accessed: 2024-07-29.
- Guo, T.; Chen, X.; Wang, Y.; Chang, R.; Pei, S.; Chawla, N. V.; Wiest, O.; and Zhang, X. 2024. Large Language Model based Multi-agents: A Survey of Progress and Challenges. *arXiv preprint arXiv:2402.01680*.
- Han, H.; Wang, Y.; Shomer, H.; Guo, K.; Ding, J.; Lei, Y.; Halappanavar, M.; Rossi, R. A.; Mukherjee, S.; Tang, X.; et al. 2024. Retrieval-augmented Generation with Graphs (GraphRAG). *arXiv preprint arXiv:2501.00309*.
- He, B.; Chen, N.; He, X.; Yan, L.; Wei, Z.; Luo, J.; and Ling, Z.-H. 2024. Retrieving, Rethinking and Revising: The Chain-of-Verification can Improve Retrieval Augmented Generation. *arXiv preprint arXiv:2410.05801*.
- Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Wang, J.; Zhang, C.; Wang, Z.; Yau, S. K. S.; Lin, Z.; et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*.
- Ji, Y.; Wu, K.; Li, J.; Chen, W.; Zhong, M.; Jia, X.; and Zhang, M. 2024. Retrieval and Reasoning on KGs: Integrate Knowledge Graphs into Large Language Models for Complex Question Answering. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 7598–7610.
- Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y. J.; Madotto, A.; and Fung, P. 2023. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12): 1–38.
- LangGraph Contributors. 2024. LangGraph: Multi-Agent Workflow Framework for LLMs. <https://www.langgraph.dev>. Accessed: 2024-07-29.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; et al. 2020. Retrieval-augmented Generation for Knowledge-intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33: 9459–9474.
- Li, G.; Hammoud, H.; Itani, H.; Khizbullin, D.; and Ghanem, B. 2023. Camel: Communicative agents for” mind” exploration of large language model society. *Advances in Neural Information Processing Systems*, 36: 51991–52008.
- Liu, A.; Feng, B.; Xue, B.; Wang, B.; Wu, B.; Lu, C.; Zhao, C.; Deng, C.; Zhang, C.; Ruan, C.; et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- Meyer, L.-P.; Frey, J.; Brei, F.; and Arndt, N. 2024. Assessing SPARQL capabilities of large language models. *arXiv preprint arXiv:2409.05925*.
- Min, B.; Ross, H.; Sulem, E.; Veyseh, A. P. B.; Nguyen, T. H.; Sainz, O.; Agirre, E.; Heintz, I.; and Roth, D. 2023. Recent Advances in Natural Language Processing via Large Pre-trained Language Models: A Survey. *ACM Computing Surveys*, 56(2): 1–40.
- OpenAI. 2024. GPT-4o Technical Report. <https://openai.com/index/gpt-4o>. OpenAI Technical Report.
- Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N. A.; and Lewis, M. 2022. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.
- Qwen Team. 2024. Qwen3: A Family of Open-Source Language Models by Alibaba Cloud. <https://github.com/QwenLM/Qwen>. Alibaba Cloud, Technical Report.
- Saxena, A.; Tripathi, A.; and Talukdar, P. 2020. Improving Multi-Hop Question Answering over Knowledge Graphs Using Knowledge Base Embeddings. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4498–4507.
- Touvron, H.; et al. 2024. LLaMA 3: Open Foundation and Instruction Models. <https://ai.meta.com/blog/meta-llama-3>. Meta AI.
- Tran, K.-T.; Dao, D.; Nguyen, M.-D.; Pham, Q.-V.; O’Sullivan, B.; and Nguyen, H. D. 2025. Multi-agent Collaboration Mechanisms: A Survey of LLMs. *arXiv preprint arXiv:2501.06322*.
- W3C. 2013. SPARQL 1.1 Query Language. <https://www.w3.org/TR/sparql11-query/>. W3C Recommendation.
- Wang, F.; Wan, X.; Sun, R.; Chen, J.; and Arık, S. Ö. 2024. Astute RAG: Overcoming Imperfect Retrieval Augmentation and Knowledge Conflicts for Large Language Models. *arXiv preprint arXiv:2410.07176*.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837.
- Wu, J.; Zhu, J.; Qi, Y.; Chen, J.; Xu, M.; Menolascina, F.; and Grau, V. 2024a. Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-augmented Generation. *arXiv preprint arXiv:2408.04187*.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2024b. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.

Wu, R.; Lee, Y.; Shu, F.; Xu, D.; Hwang, S.-w.; Yao, Z.; He, Y.; and Yan, F. 2025. ComposeRAG: A Modular and Composable RAG for Corpus-Grounded Multi-Hop Question Answering. *arXiv preprint arXiv:2506.00232*.

Xu, S.; Pang, L.; Shen, H.; Cheng, X.; and Chua, T.-S. 2024. Search-in-the-Chain: Interactively Enhancing Large Language Models with Search for Knowledge-intensive Tasks. In *Proceedings of the ACM Web Conference 2024*, 1362–1373.

Yang, Y.; Chai, H.; Shao, S.; Song, Y.; Qi, S.; Rui, R.; and Zhang, W. 2025. Agentnet: Decentralized Evolutionary Coordination for LLM-based Multi-agent Systems. *arXiv preprint arXiv:2504.00587*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Yasunaga, M.; Ren, H.; Bosselut, A.; Liang, P.; and Leskovec, J. 2021. QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 535–546.

Yih, S. W.-t.; Chang, M.-W.; He, X.; and Gao, J. 2015. Semantic Parsing via Staged Query Graph Generation: Question Answering with Knowledge Base. In *Proceedings of the Joint Conference of the 53rd Annual Meeting of the ACL and the 7th International Joint Conference on Natural Language Processing of the AFNLP*.

Zhao, X.; Gao, F.; Song, X.; Chen, Y.; Yang, R.; Fu, Y.; Wang, Y.; Iwasawa, Y.; Matsuo, Y.; and Li, I. 2025. ReAgent: Reversible Multi-Agent Reasoning for Knowledge-Enhanced Multi-Hop QA. *arXiv:2503.06951*.